Introduction to Data Mining

Lab 3 – Simple Classifiers

3.1. Simplicity first!

In the third class, we are going to learn how to examine some data mining algorithms on datasets using Weka. (See the lecture of class 3 by Ian H. Witten, [1][1])

In this section, we learn how **OneR** (one attribute does all the work) works. Open weather.nominal.arff, run OneR, look at the classifier model, how is it?

```
Correctly Classified Instances          10               71.4286 %
Incorrectly Classified Instances         4               28.5714 %
Kappa statistic                          0.3778
Mean absolute error                      0.2857
Root mean squared error                  0.5345
Relative absolute error                 61.5385 %
Root relative squared error            111.4773 %
Total Number of Instances               14

=== Detailed Accuracy By Class ===

                 TP Rate   FP Rate   Precision   Recall   F-Measure   MCC      ROC Area
PRC Area  Class
                 0.778     0.400     0.778       0.778    0.778       0.378    0.689
0.748     yes
                 0.600     0.222     0.600       0.600    0.600       0.378    0.689
0.503     no
Weighted Avg.    0.714     0.337     0.714       0.714    0.714       0.378    0.689
0.660

=== Confusion Matrix ===

 a b   <-- classified as
 7 2 | a = yes
 2 3 | b = no
```

- Remarks: Correctly Classified Instances - 10 - 71.4286%

Use OneR to build decision tree for some datasets. Compared with ZeroR, how does OneR perform?

[1] http://www.cs.waikato.ac.nz/ml/weka/mooc/dataminingwithweka/

Dataset	OneR - accuracy	ZeroR - accuracy
weather.nominal.arff	71.4286%	64.2857%
weather.numeric.arff	71.4286%	64.2857%
glass.arff	63.0841%	35.514%
iriss.arff	96%	33.3333%
labor.arff	84.2105%	64.9123%
diabetes.arff	76.4323%	65.1042%
vote.arff	95.6322%	61.3793%
soybean.arff	40.8492%	13.47%
breast-cancer.arff	72.7273%	70.2797%
credit-g.arff	74.3%	70%

3.2. Overfitting

What is "overfitting"? - **overfitting** occurs when a statistical model describes **random error** or **noise** instead of the underlying relationship, b/c of complex model, noise/error in the data, or unsuitable applied criterion, → poor prediction. To avoid this, use cross-validation, or pruning... [ref: http://en.wikipedia.org/wiki/Overfitting]

Follow the instructions in [1], run OneR on the weather.numeric and diabetes dataset…

Write down the results in the following table: (cross-validation used)

Dataset	OneR	ZeroR
weather.numeric	Classifier model: outlook: sunny → no, overcast → yes, rainy → yes (10/14 training correct) Accuracy: 42.8571%	Classifier model: Always predicts yes Accuracy: 64.2857%
weather.numeric w/o outlook att.	Classifier model: humidity: < 82.5 → yes, ≥ 82.5 → no (10/14 training correct) Accuracy: 50%	Classifier model: Always predicts yes Accuracy: 64.2857%
diabetes	Classifier model: splits on plas (10 thresholds, ~587/768 correct) Accuracy: 57.1615%	Classifier model: Always predicts tested_negative Accuracy: 65.1042%
Diabetes w/ minBucketSize 1	Classifier model: splits on pedi (200+ micro-thresholds, zigzags yes/no) Accuracy: 57.1615%	Classifier model: Always predicts tested_negative Accuracy: 65.1042%

MinBucketSize? - It sets the minimum number of training instances that a bucket (value interval) must contain. Smaller values = more specific rules = risk of overfitting. Larger values = broader rules = less overfitting.

Remark? - When minBucketSize is very small (=1), OneR creates very specific rules that fit the training data too closely → the 10-fold CV accuracy may be lower than with default settings, demonstrating overfitting.

3.3. Using probabilities

Lecture of Naïve Bayes: [1]

➔ All attributes contribute equally and independently → no identical attributes

Follow the instructions in [1] to exame **NaiveBayes** on *weather.nominal*

Classifier model	Performance
Naive Bayes Classifier Class Attribute yes no (0.63) (0.38) ========== outlook sunny 3.0 4.0 overcast 5.0 1.0 rainy 4.0 3.0 [total] 12.0 8.0 temperature hot 3.0 3.0 mild 5.0 3.0 cool 4.0 2.0 [total] 12.0 8.0 humidity high 4.0 5.0 normal 7.0 2.0 [total] 11.0 7.0 windy TRUE 4.0 4.0 FALSE 7.0 3.0 [total] 11.0 7.0	92.8571%

3.4. Decision Trees

Lecture of decision trees: [1]

How to calculate entropy and information gain?

Entropy measures the impurity of a collection.

$$Entropy(S) = -\sum_{i=1}^{c} p_i log_2 p_i$$

Information Gain measures the Expected Reduction in Entropy.

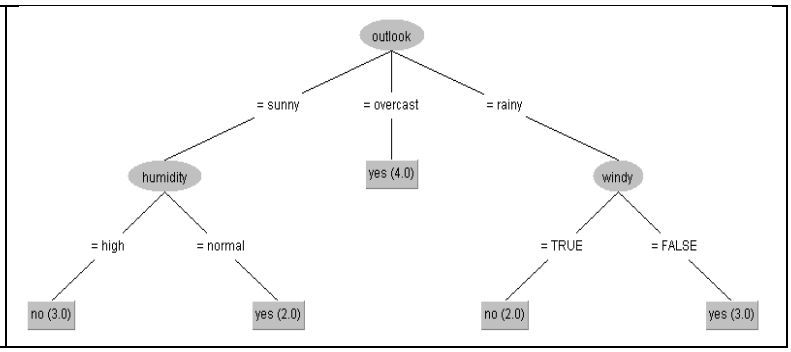
Info. Gain = (Entropy of distribution before the split) – (Entropy of distribution after the split)

$$Gain(S, A) \equiv Entropy(S) - \sum_{v \in Values(A)} \frac{|S_v|}{|S|} Entropy(S_v)$$

Values(*A*) is the set of all possible values for attribute *A* and S_v is the subset of *S* for which attribute *A* has value.

Build a decision tree for the weather data step by step:

Compute Entropy and Info. Gain	**Selected attribute**
All 14 instances, 9 yes / 5 no: Entropy(S) = 0.940. Gain(outlook) = 0.247, Gain(humidity) = 0.152, Gain(windy) = 0.048, Gain(temp) = 0.029	outlook
Sunny branch (5 instances, 2 yes / 3 no): Entropy = 0.971. Gain(humidity) = 0.971, Gain(temp) = 0.571, Gain(windy) = 0.020. Rainy branch (5 instances, 3 yes / 2 no): Entropy = 0.971. Gain(windy) = 0.971, Gain(temp) = 0.020, Gain(humidity) = 0.020. Overcast → pure leaf (all yes, entropy=0)	humidity (sunny) windy (rainy)
All remaining branches are pure leaves (entropy = 0). Tree is complete. Sunny + High → no, Sunny + Normal → yes, Rainy + FALSE → yes, Rainy + TRUE → no	*(no more splits needed)*



Use Weka to examine J48 on the weather data.

3.5. Pruning decision trees

Follow the lecture of pruning decision tree in [1] …

Why pruning? - Prevent overfitting to noise in the data.

In Weka, look at the J48 leaner. What are parameters: minNunObj, confidenceFactor?

- minNumObj: Minimum number of instances a leaf must have. Higher = simpler tree (less overfitting).
- confidenceFactor: Controls pruning aggressiveness. Lower value (e.g., 0.1) = more pruning = simpler tree. Default = 0.25.

Follow the instructions in [1] to run J48 on the two dataset, then fill in the following table:

Dataset	J48 (default, pruned)	J48 (unpruned)
diabetes.arff	73.8281%	72.6563%
breast-cancer.arff	75.5245%	69.5804%

3.6. Nearest neighbor

Follow the lecture in [1]

“Instance-based” learning = “nearest-neighbor” learning

What is k-nearest-neighbors (K-NN)?

– KNN classifies a new instance by finding the K closest training instances (neighbors) and using a majority vote. No model is built — classification happens at prediction time (lazy learning).

Follow the instructions in [1] to run lazy>IBk on the *glass* dataset with k = 1, 5, 20, and then fill its accuracy in the following table:

Dataset	**IBk, k =1**	**IBk, k =5**	**IBk, k =20**
Glass	70.5607%	67.757%	65.4206%